Pràctica_1

March 4, 2026

[ ]:

```

```

1 Practice 1

1.1 Statistical learning with deep artificial neural networks

1.2 Introduction

In this activity we propose the implementation of a dense deep neural network for prediction of the severity of Parkinson's disease from data captured by telemonitoring of the voice (16 variables) of 42 patients. Additional information about the data is can be found at: https://archive.ics.uci.edu/ml/datasets/Parkinsons+Telemonitoring.

2 Solution

1. Cargar el fichero de datos parkinsons_updrs.data.

[1]:

```
import pandas as pd
import numpy as np
file_path = "parkinsons_updrs.data"
dat = pd.read_csv(file_path, header=0, sep=",", encoding="latin1") # el 0␣
↪equivale a TRUE, en caso contrario se pone None
```

2. Realizar un descriptiva de las variables del dataset.

[2]:

```
dat.shape
```

[2]:

```
(5875, 22)
```

[3]:

```
dat.head()
```

[3]:

```
   subject#  age  sex  test_time  motor_UPDRS  total_UPDRS  Jitter(%)  \
0         1   72    0     5.6431       28.199       34.398    0.00662
1         1   72    0    12.6660       28.447       34.894    0.00300
2         1   72    0    19.6810       28.695       35.389    0.00481
3         1   72    0    25.6470       28.905       35.810    0.00528
4         1   72    0    33.6420       29.187       36.375    0.00335

   Jitter(Abs)  Jitter:RAP  Jitter:PPQ5  …  Shimmer(dB)  Shimmer:APQ3  \
```

```
0     0.000034     0.00401      0.00317  …        0.230       0.01438
1     0.000017     0.00132      0.00150  …        0.179       0.00994
2     0.000025     0.00205      0.00208  …        0.181       0.00734
3     0.000027     0.00191      0.00264  …        0.327       0.01106
4     0.000020     0.00093      0.00130  …        0.176       0.00679

   Shimmer:APQ5  Shimmer:APQ11  Shimmer:DDA       NHR     HNR     RPDE  \
0       0.01309        0.01662      0.04314  0.014290  21.640  0.41888
1       0.01072        0.01689      0.02982  0.011112  27.183  0.43493
2       0.00844        0.01458      0.02202  0.020220  23.047  0.46222
3       0.01265        0.01963      0.03317  0.027837  24.445  0.48730
4       0.00929        0.01819      0.02036  0.011625  26.126  0.47188

       DFA      PPE
0  0.54842  0.16006
1  0.56477  0.10810
2  0.54405  0.21014
3  0.57794  0.33277
4  0.56122  0.19361

[5 rows x 22 columns]
```

[4]:
```python
dat["sex"] = dat["sex"].astype("category") # lo pasa a factor
dat.dtypes # equivalente a str()
```

[4]:
```
subject#            int64
age                 int64
sex              category
test_time         float64
motor_UPDRS       float64
total_UPDRS       float64
Jitter(%)         float64
Jitter(Abs)       float64
Jitter:RAP        float64
Jitter:PPQ5       float64
Jitter:DDP        float64
Shimmer           float64
Shimmer(dB)       float64
Shimmer:APQ3      float64
Shimmer:APQ5      float64
Shimmer:APQ11     float64
Shimmer:DDA       float64
NHR               float64
HNR               float64
RPDE              float64
DFA               float64
PPE               float64
```

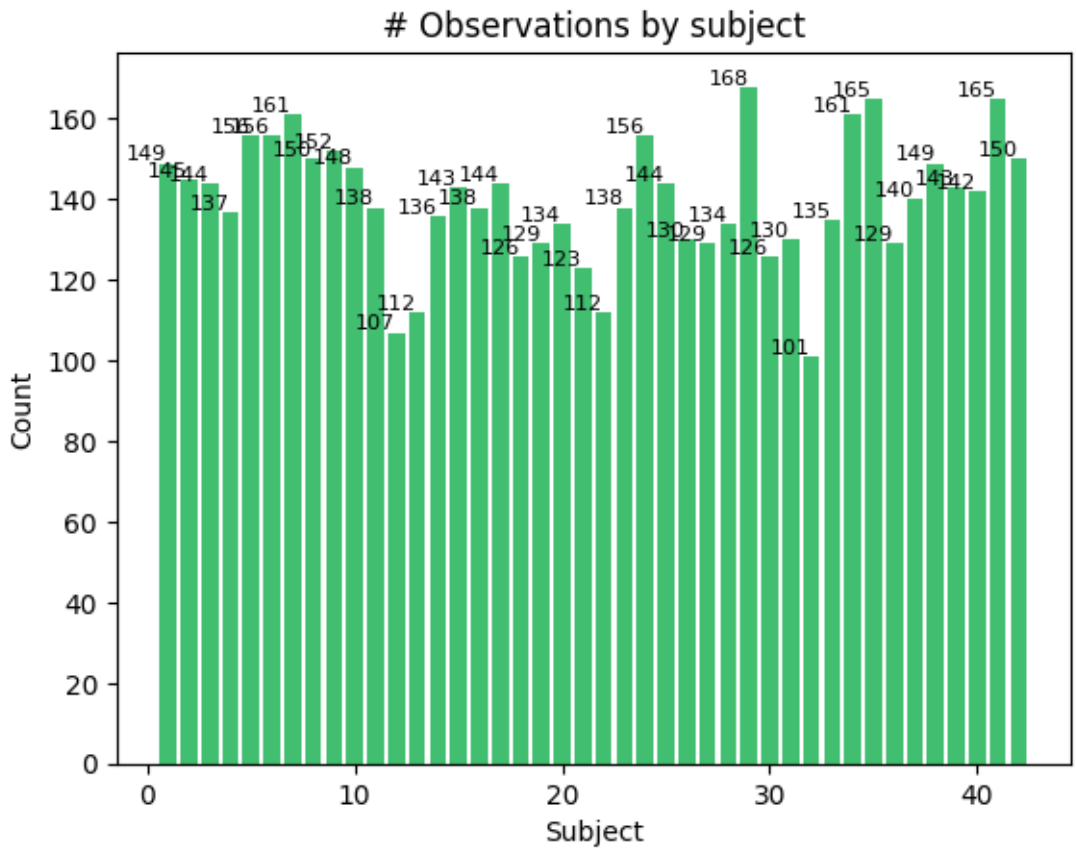
```
dtype: object
```

[5]:
```python
# Univariante
summary = dat.describe(include='all')
print(summary)
```

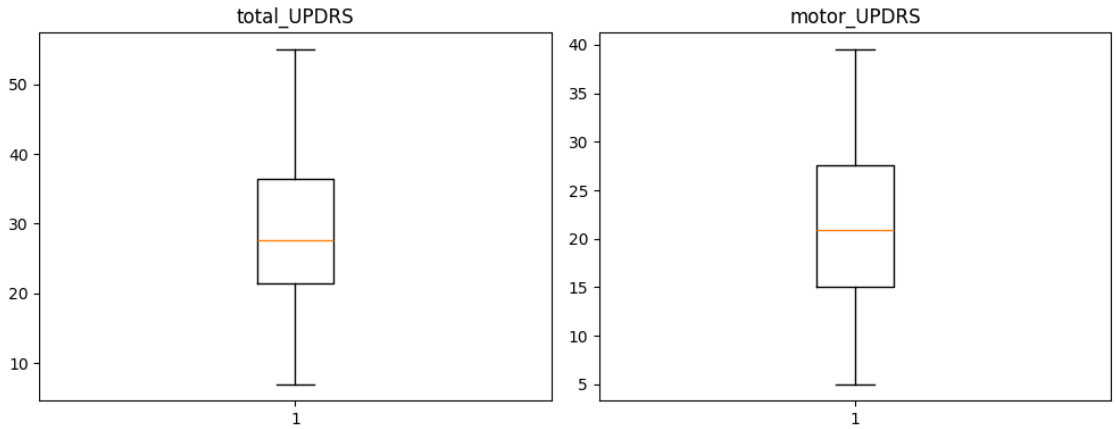
```
           subject#          age     sex    test_time  motor_UPDRS  \
count   5875.000000  5875.000000  5875.0  5875.000000  5875.000000
unique          NaN          NaN     2.0          NaN          NaN
top             NaN          NaN     0.0          NaN          NaN
freq            NaN          NaN  4008.0          NaN          NaN
mean      21.494128    64.804936     NaN    92.863722    21.296229
std       12.372279     8.821524     NaN    53.445602     8.129282
min        1.000000    36.000000     NaN    -4.262500     5.037700
25%       10.000000    58.000000     NaN    46.847500    15.000000
50%       22.000000    65.000000     NaN    91.523000    20.871000
75%       33.000000    72.000000     NaN   138.445000    27.596500
max       42.000000    85.000000     NaN   215.490000    39.511000

        total_UPDRS    Jitter(%)  Jitter(Abs)   Jitter:RAP  Jitter:PPQ5  …  \
count   5875.000000  5875.000000  5875.000000  5875.000000  5875.000000  …
unique          NaN          NaN          NaN          NaN          NaN  …
top             NaN          NaN          NaN          NaN          NaN  …
freq            NaN          NaN          NaN          NaN          NaN  …
mean      29.018942     0.006154     0.000044     0.002987     0.003277  …
std       10.700283     0.005624     0.000036     0.003124     0.003732  …
min        7.000000     0.000830     0.000002     0.000330     0.000430  …
25%       21.371000     0.003580     0.000022     0.001580     0.001820  …
50%       27.576000     0.004900     0.000035     0.002250     0.002490  …
75%       36.399000     0.006800     0.000053     0.003290     0.003460  …
max       54.992000     0.099990     0.000446     0.057540     0.069560  …

        Shimmer(dB)  Shimmer:APQ3  Shimmer:APQ5  Shimmer:APQ11  Shimmer:DDA  \
count   5875.000000   5875.000000   5875.000000    5875.000000  5875.000000
unique          NaN           NaN           NaN            NaN          NaN
top             NaN           NaN           NaN            NaN          NaN
freq            NaN           NaN           NaN            NaN          NaN
mean       0.310960      0.017156      0.020144       0.027481     0.051467
std        0.230254      0.013237      0.016664       0.019986     0.039711
min        0.026000      0.001610      0.001940       0.002490     0.004840
25%        0.175000      0.009280      0.010790       0.015665     0.027830
50%        0.253000      0.013700      0.015940       0.022710     0.041110
75%        0.365000      0.020575      0.023755       0.032715     0.061735
max        2.107000      0.162670      0.167020       0.275460     0.488020

                NHR          HNR         RPDE          DFA          PPE
count   5875.000000  5875.000000  5875.000000  5875.000000  5875.000000
unique          NaN          NaN          NaN          NaN          NaN
```

```
top             NaN          NaN          NaN          NaN          NaN
freq            NaN          NaN          NaN          NaN          NaN
mean       0.032120    21.679495     0.541473     0.653240     0.219589
std        0.059692     4.291096     0.100986     0.070902     0.091498
min        0.000286     1.659000     0.151020     0.514040     0.021983
25%        0.010955    19.406000     0.469785     0.596180     0.156340
50%        0.018448    21.920000     0.542250     0.643600     0.205500
75%        0.031463    24.444000     0.614045     0.711335     0.264490
max        0.748260    37.875000     0.966080     0.865600     0.731730

[11 rows x 22 columns]
```

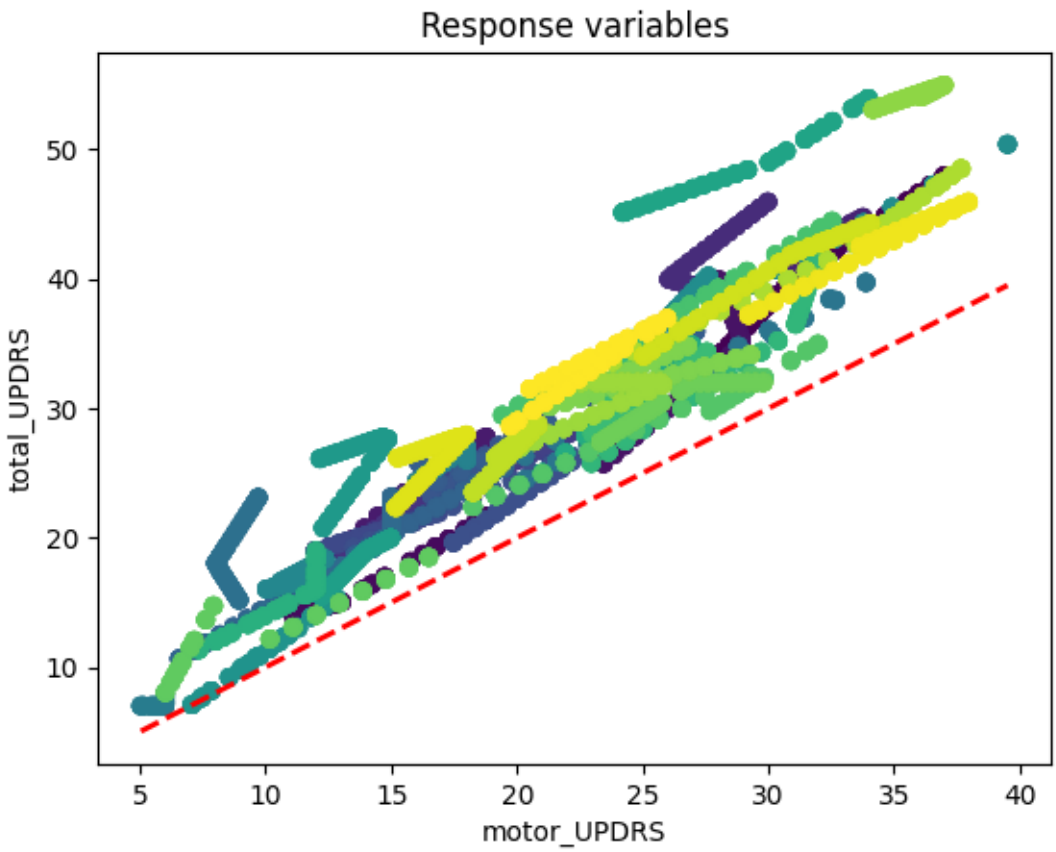
[6]:
```python
import matplotlib.pyplot as plt
tt = dat['subject#'].value_counts().sort_index() # cuenta las veces de la␣
↪variable subject i lo ordena
bars = plt.bar(tt.index, tt.values, color=plt.cm.viridis(dat['age'].
↪astype('category').cat.codes / dat['age'].nunique()))
# el index son las etiquetas i el values son las freq.
plt.title("# Observations by subject")
plt.xlabel("Subject")
plt.ylabel("Count")

for i, value in enumerate(tt.values):
    plt.text(i, value - 0.3, str(value), ha='center', va='bottom', fontsize=8)

plt.show()
```



[7]:
```
plt.figure(figsize=(10, 4)) # define el alto y ancho de la figura

plt.subplot(1, 2, 1) # equivalente a par(mfrow=c()), una fila dos columnas␣
 ↪posicion 1
plt.boxplot(dat['total_UPDRS'])
plt.title("total_UPDRS")

plt.subplot(1, 2, 2) # posicion 2
plt.boxplot(dat['motor_UPDRS'])
plt.title("motor_UPDRS")

plt.tight_layout()
plt.show()
```



[8]:
```
# Bivariante
subjects = dat['subject#'].astype('category').cat.codes  # convierte sujetos a␣
↪códigos numéricos (0,1,2,...)
# Crear gráfico de dispersión
plt.scatter(
    dat['motor_UPDRS'],       # eje X
    dat['total_UPDRS'],       # eje Y
    c=dat['subject#'].astype('category').cat.codes,  # colores por sujeto
    cmap='viridis'            # paleta de colores
)

# Título y etiquetas
plt.title("Response variables")
plt.xlabel("motor_UPDRS")
plt.ylabel("total_UPDRS")

# Añadir línea y = x (equivalente a abline(a=0,b=1,...))
plt.plot(
    [dat['motor_UPDRS'].min(), dat['motor_UPDRS'].max()],
    [dat['motor_UPDRS'].min(), dat['motor_UPDRS'].max()],
    color='red', linestyle='--', linewidth=2
)

plt.show()
```



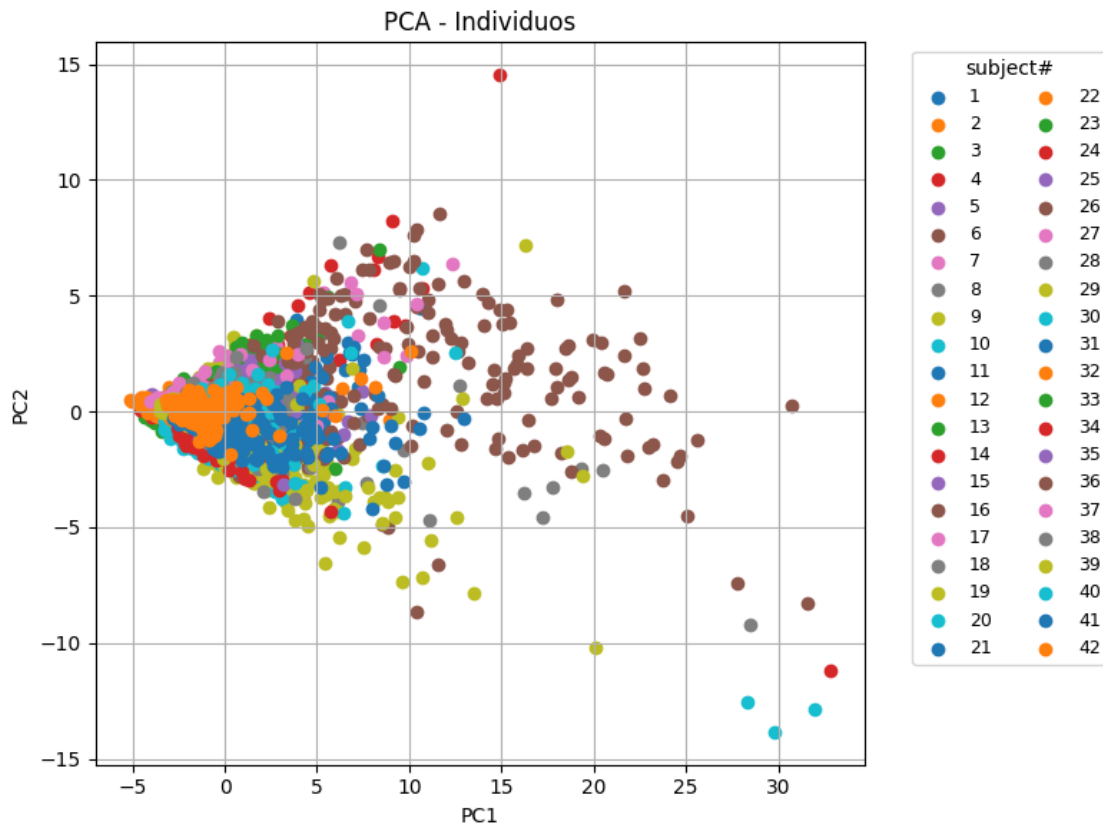
[9]:
```
# PCA
import prince
# Convertir la variable cualitativa a string
dat['subject#'] = dat['subject#'].astype(str)

# Seleccionar columnas 1 y 7-22 (Python indexa desde 0)
X = dat.iloc[:, [0] + list(range(6, 22))]

# Quitar la variable cualitativa para el PCA
X_quant = X.drop(X.columns[0], axis=1)

# PCA con todas las componentes posibles
pca = prince.PCA(n_components=X_quant.shape[1]).fit(X_quant)

# Obtener coordenadas de variables (correlaciones con cada componente)
var_coords = pca.column_correlations

# Calcular contribución de cada variable a cada componente
# Fórmula: (coord^2 / sum(coord^2 para esa componente) * 100)
```

```python
var_contrib = var_coords**2
var_contrib = var_contrib.div(var_contrib.sum(axis=0), axis=1) * 100

# Mostrar resultados, con 6 decimales
pd.set_option('display.float_format', '{:.6f}'.format)
print(var_contrib)
```

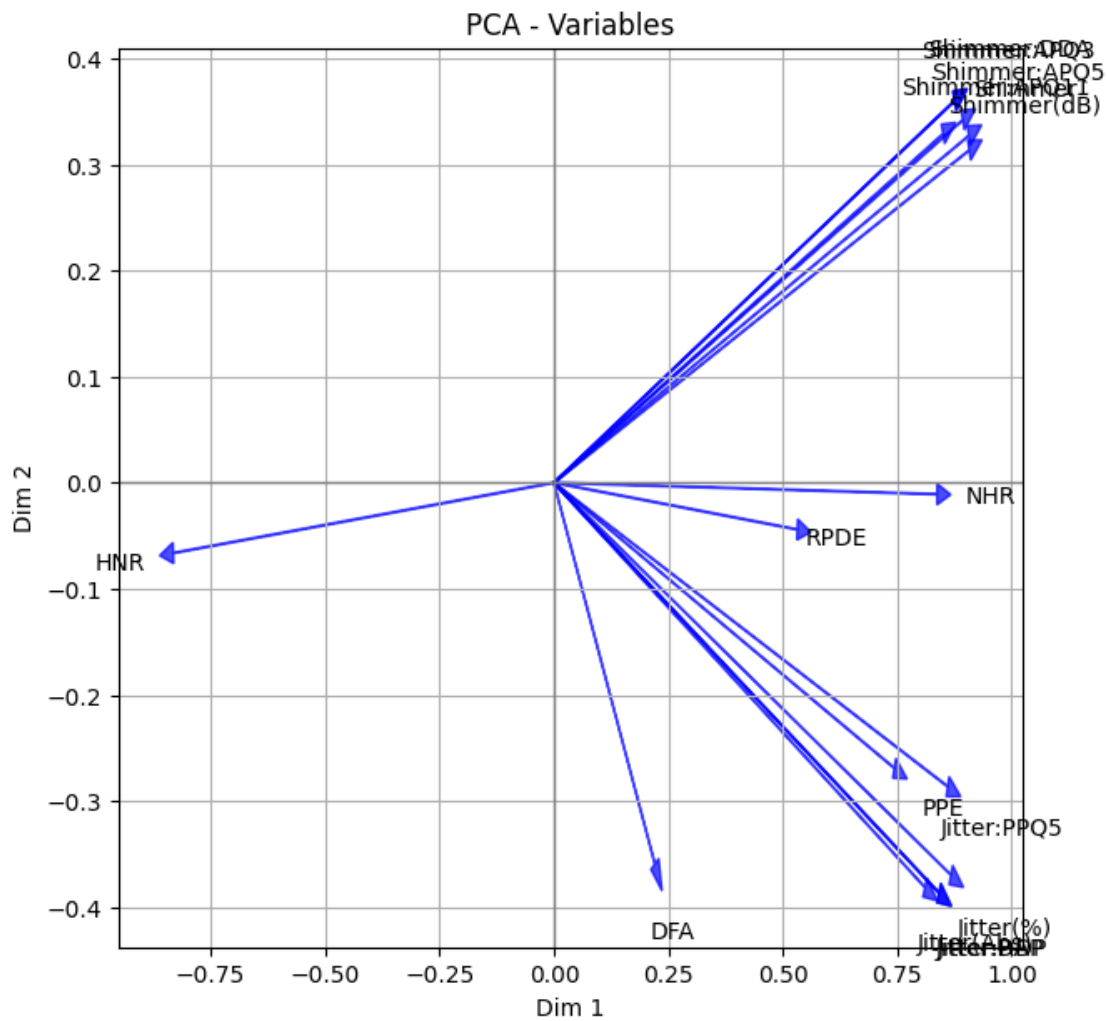
```
component           0        1         2         3         4         5   \
variable
Jitter(%)     7.111863 8.693309  3.324776  0.025894  0.191971  0.011983
Jitter(Abs)   6.231562 9.282177  0.634415  0.584192  1.206865 18.059313
Jitter:RAP    6.698493 9.521303  4.791887  0.274463  1.090298  2.567831
Jitter:PPQ5   7.014391 5.229241  5.790529  0.175404  1.091928  6.930681
Jitter:DDP    6.698565 9.521363  4.791616  0.274595  1.089921  2.566602
Shimmer       7.757040 6.796129  0.000159  0.770057  0.078459  0.259315
Shimmer(dB)   7.773366 6.238917  0.000151  0.492908  0.057719  0.073512
Shimmer:APQ3  7.241287 8.238801  0.012946  1.572819  0.115746 10.151874
Shimmer:APQ5  7.535111 7.424208  0.009518  0.969306  0.253174  0.814125
Shimmer:APQ11 6.845996 6.889057  0.833981  0.645095  0.213563  5.803872
Shimmer:DDA   7.241264 8.238869  0.012944  1.572982  0.115717 10.151898
NHR           6.676533 0.007484  7.658650  1.940691  1.384422 31.505782
HNR           6.587884 0.281540  7.150962  3.026245  2.374720  6.810884
RPDE          2.800176 0.129333 17.777697 50.249049 21.054269  0.657661
DFA           0.495626 8.849434 38.533450 36.144938 10.265582  3.102008
PPE           5.290845 4.658834  8.676318  1.281363 59.415646  0.532659

component            6         7         8         9         10        11   \
variable
Jitter(%)      1.225656  1.516076  0.016902  0.188196  2.127584  0.033994
Jitter(Abs)    3.299822 36.044411 20.349197  3.744635  0.027194  0.006329
Jitter:RAP     0.040400  6.531304  0.028595  9.378430  0.912198  2.261688
Jitter:PPQ5    1.569009  1.816018  1.141389 49.337953  0.728118  9.963997
Jitter:DDP     0.040364  6.530900  0.028695  9.376283  0.909733  2.254034
Shimmer        0.220803  0.225490  0.370389  0.007464  7.256669  0.195458
Shimmer(dB)    0.835391  0.331803  0.139786  1.512090 59.472183  2.485776
Shimmer:APQ3   1.892870  0.049285  5.831678  0.000074  4.435949  8.453857
Shimmer:APQ5   0.628891  0.368031  0.006384 11.126427 12.715205 56.671094
Shimmer:APQ11 24.556031  0.379087 37.123334  3.588048  4.412131  8.360148
Shimmer:DDA    1.893092  0.049314  5.831664  0.000071  4.435855  8.451757
NHR            5.200347 27.717751  4.550127 10.856970  1.599448  0.820171
HNR           47.572851 17.431805  8.277315  0.171212  0.251099  0.003760
RPDE           5.001750  0.090828  2.198110  0.000068  0.011826  0.000104
DFA            0.041113  0.905777  1.277395  0.360364  0.011079  0.005728
PPE            5.981610  0.012121 12.829040  0.351715  0.693728  0.032104

component           12        13        14        15
variable
Jitter(%)     75.175715  0.356082  0.000000  0.000000
```

```
Jitter(Abs)     0.527461  0.002425  0.000000  0.000000
Jitter:RAP      5.906507  0.002109 49.993940  0.000552
Jitter:PPQ5     8.912941  0.298400  0.000001  0.000000
Jitter:DDP      5.909755  0.002070 50.004958  0.000545
Shimmer         1.058024 75.004543  0.000000  0.000000
Shimmer(dB)     0.885826 19.700571  0.000000  0.000000
Shimmer:APQ3    0.423998  1.578712  0.000555 49.999549
Shimmer:APQ5    0.459780  1.018743  0.000003  0.000000
Shimmer:APQ11   0.004321  0.345337  0.000000  0.000000
Shimmer:DDA     0.424671  1.580006  0.000542 49.999353
NHR             0.006269  0.075355  0.000000  0.000000
HNR             0.053894  0.005829  0.000000  0.000000
RPDE            0.020274  0.008855  0.000000  0.000000
DFA             0.000423  0.007083  0.000000  0.000000
PPE             0.230139  0.013878  0.000000  0.000000
```

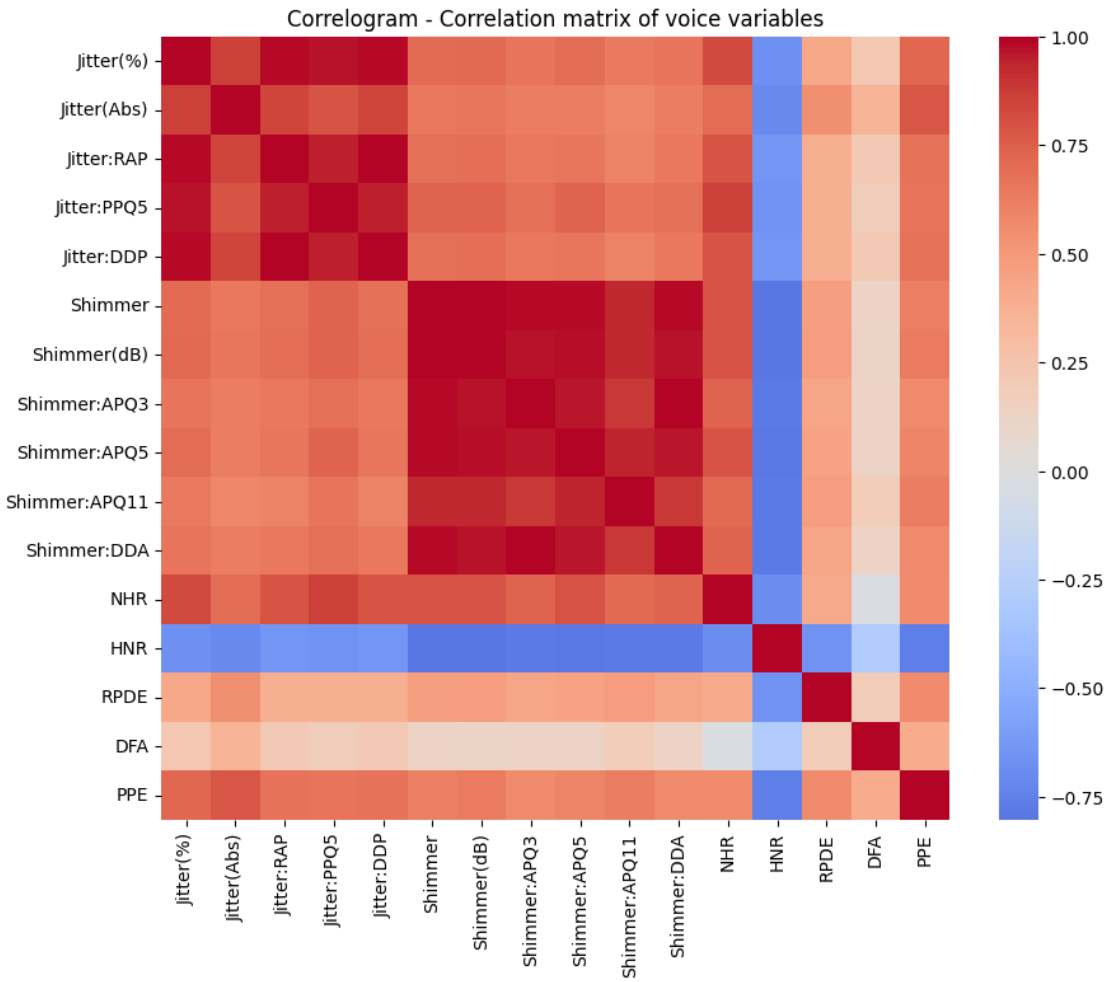
[10]:
```python
ind_coords = pca.row_coordinates(X)
plt.figure(figsize=(8,6))
for cat in dat['subject#'].unique():
    subset = ind_coords[dat['subject#'] == cat]
    plt.scatter(subset[0], subset[1], label=cat)

plt.xlabel('PC1')
plt.ylabel('PC2')
plt.title('PCA - Individuos')
plt.grid(True)

# Leyenda en dos columnas, fuera del gráfico
plt.legend(title='subject#', ncol=2, fontsize=9, bbox_to_anchor=(1.05, 1),␣
 ↪loc='upper left')

plt.tight_layout()  # Ajusta el gráfico para que no se corte la leyenda
plt.show()
```



[11]:
```
# Coordenadas de las variables en el plano factorial
var_coords = pca.column_correlations

plt.figure(figsize=(7,7))
plt.axhline(0, color='gray', lw=1)
plt.axvline(0, color='gray', lw=1)

# Dibujar flechas para cada variable
for i in range(var_coords.shape[0]):
    plt.arrow(0, 0, var_coords.iloc[i, 0], var_coords.iloc[i, 1],
              color='blue', alpha=0.7, head_width=0.02,␣
 ↪length_includes_head=True)
    plt.text(var_coords.iloc[i, 0]*1.1, var_coords.iloc[i, 1]*1.1,
             var_coords.index[i], color='black', ha='center', va='center')

plt.xlabel("Dim 1")
plt.ylabel("Dim 2")
plt.title("PCA - Variables")
plt.grid(True)
plt.show()
```



[12]:
```
# Matriz de correlación
import seaborn as sns
X_voice = dat.iloc[:, 6:22]
plt.figure(figsize=(10,8))
sns.heatmap(X_voice.corr(), annot=False, cmap='coolwarm', center=0)
plt.title("Correlogram - Correlation matrix of voice variables")
plt.show()
```



3. Normalizar mediante la transformación min-max las variables de las 16 medidas de voz.

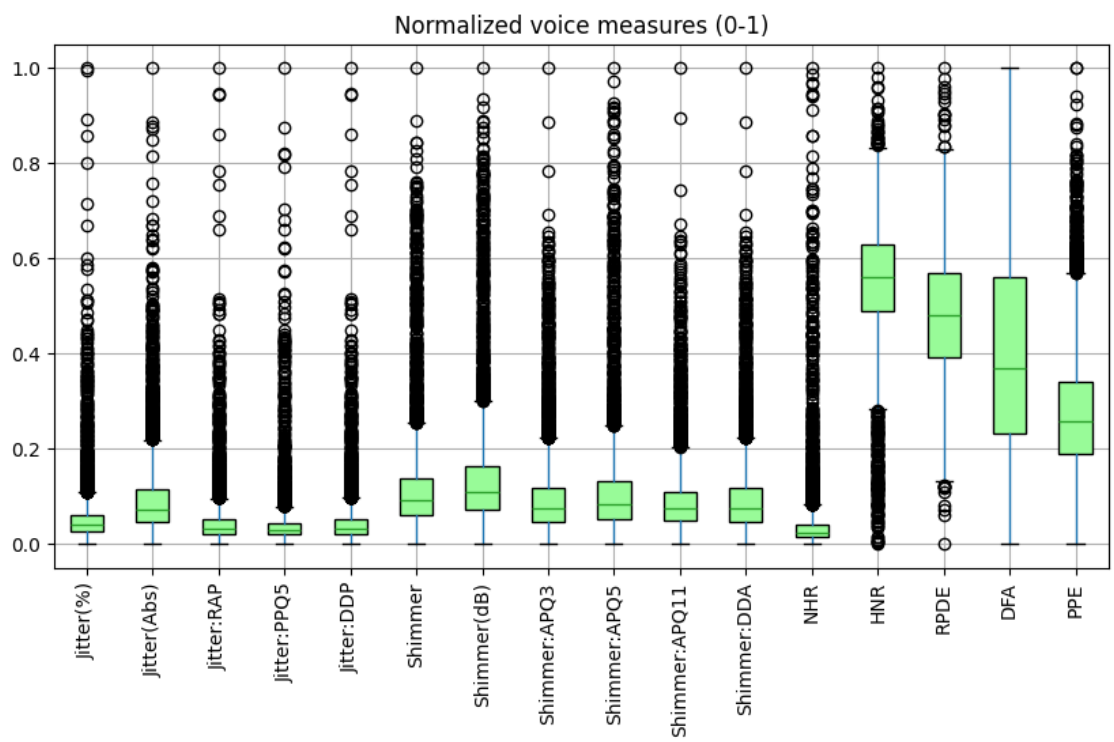
[13]:
```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
dat_nrm = pd.DataFrame(scaler.fit_transform(X_voice), columns=X_voice.columns)␣
↪# transform aplica la fórmula (x - min)/(max - min) a cada valor.
print(dat_nrm.describe())
# Boxplot de los datos normalizados
plt.figure(figsize=(10,5))
dat_nrm.boxplot(rot=90, patch_artist=True,
                boxprops=dict(facecolor='palegreen'))
plt.title("Normalized voice measures (0-1)")
plt.show()
```

```
        Jitter(%)  Jitter(Abs)  Jitter:RAP  Jitter:PPQ5  Jitter:DDP  \
count 5875.000000  5875.000000 5875.000000  5875.000000 5875.000000
mean     0.053689     0.094233    0.046446     0.041181    0.046500
```

std	0.056719	0.081164	0.054603	0.053979	0.054597
min	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.027733	0.045529	0.021849	0.020107	0.021847
50%	0.041045	0.072811	0.033561	0.029799	0.033615
75%	0.060206	0.115228	0.051739	0.043830	0.051791
max	1.000000	1.000000	1.000000	1.000000	1.000000

	Shimmer	Shimmer(dB)	Shimmer:APQ3	Shimmer:APQ5	Shimmer:APQ11	\
count	5875.000000	5875.000000	5875.000000	5875.000000	5875.000000	
mean	0.116637	0.136934	0.096521	0.110275	0.091552	
std	0.097282	0.110646	0.082187	0.100945	0.073217	
min	0.000000	0.000000	0.000000	0.000000	0.000000	
25%	0.060474	0.071600	0.047622	0.053610	0.048265	
50%	0.092066	0.109082	0.075065	0.084807	0.074074	
75%	0.138156	0.162902	0.117751	0.132148	0.110726	
max	1.000000	1.000000	1.000000	1.000000	1.000000	

	Shimmer:DDA	NHR	HNR	RPDE	DFA	PPE
count	5875.000000	5875.000000	5875.000000	5875.000000	5875.000000	5875.000000
mean	0.096500	0.042560	0.552808	0.479048	0.395949	0.278418
std	0.082188	0.079805	0.118486	0.123900	0.201678	0.128917
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.047581	0.014264	0.490032	0.391094	0.233644	0.189303
50%	0.075065	0.024282	0.559449	0.480001	0.368529	0.258567
75%	0.117751	0.041682	0.629142	0.568087	0.561199	0.341681
max	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000



4. Crear la variable binaria de severidad del Parkinson.

[14]:
```
dat['total_sev'] = np.where(dat['total_UPDRS'] > 25, 1, 0)
print(dat['total_sev'].value_counts())
```

```
total_sev
1    3687
0    2188
Name: count, dtype: int64
```

5. Separar train y test.

[15]:
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    dat_nrm, dat['total_sev'], test_size=0.3, random_state=123
)
```

6. Implementar una red neuronal profunda densa para la predicción de la severidad con dos capas ocultas de 10 y 10 nodos cada una.

[16]:
```
import tensorflow as tf
from tensorflow.keras import Input
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.regularizers import l2

input_shape = X_train.shape[1]

model1 = Sequential([
    Input(shape=(input_shape,)),  # primera capa Input
    Dense(10, activation='relu'),  # sin input_shape aquí
    Dense(10, activation='relu'),
    Dense(1, activation='sigmoid')
])

model1.compile(optimizer='adam', loss='binary_crossentropy',␣
 ↪metrics=['accuracy'])
```

```
2026-03-04 13:17:41.116179: E
external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:477] Unable to register
cuFFT factory: Attempting to register factory for plugin cuFFT when one has
already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
E0000 00:00:1772626661.126142   20175 cuda_dnn.cc:8310] Unable to register cuDNN
factory: Attempting to register factory for plugin cuDNN when one has already
been registered
```

```
E0000 00:00:1772626661.129982   20175 cuda_blas.cc:1418] Unable to register
cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has
already been registered
2026-03-04 13:17:41.142136: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild
TensorFlow with the appropriate compiler flags.
I0000 00:00:1772626662.392447   20175 gpu_device.cc:2022] Created device
/job:localhost/replica:0/task:0/device:GPU:0 with 22314 MB memory:  -> device:
0, name: NVIDIA GeForce RTX 3090, pci bus id: 0000:01:00.0, compute capability:
8.6
```

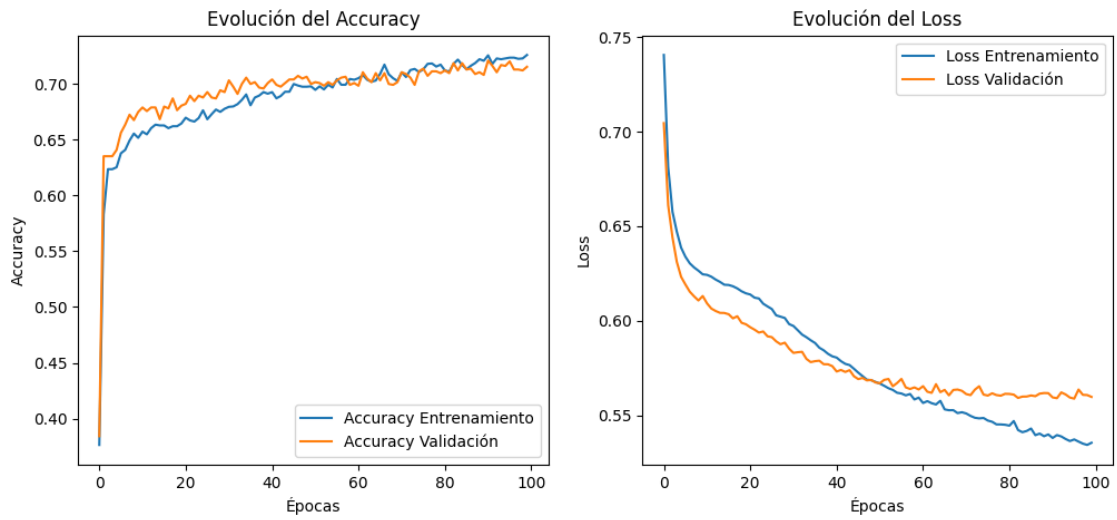
[17]:
```
# --------------------------
# 2 Entrenamiento de los modelos
# --------------------------

history1 = model1.fit(X_train, y_train, epochs=100, batch_size=32,␣
↪validation_split=0.3, verbose=0)
```

```
WARNING: All log messages before absl::InitializeLog() is called are written to
STDERR
I0000 00:00:1772626663.554513   20246 service.cc:148] XLA service 0x7396b4005ce0
initialized for platform CUDA (this does not guarantee that XLA will be used).
Devices:
I0000 00:00:1772626663.554535   20246 service.cc:156]   StreamExecutor device
(0): NVIDIA GeForce RTX 3090, Compute Capability 8.6
2026-03-04 13:17:43.570011: I
tensorflow/compiler/mlir/tensorflow/utils/dump_mlir_util.cc:268] disabling MLIR
crash reproducer, set env var `MLIR_CRASH_REPRODUCER_DIRECTORY` to enable.
I0000 00:00:1772626663.654178   20246 cuda_dnn.cc:529] Loaded cuDNN version
90800
I0000 00:00:1772626664.041846   20246 device_compiler.h:188] Compiled cluster
using XLA!  This line is logged at most once for the lifetime of the process.
```

[18]:
```
# --------------------------
# 3 Evaluación en conjunto de prueba
# --------------------------

score1 = model1.evaluate(X_test, y_test, verbose=0)
```

[19]:
```
# Accuracy
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.plot(history1.history['accuracy'], label='Accuracy Entrenamiento')
plt.plot(history1.history['val_accuracy'], label='Accuracy Validación')
plt.title('Evolución del Accuracy')
```

```
plt.xlabel('Épocas')
plt.ylabel('Accuracy')
plt.legend()

# Loss
plt.subplot(1,2,2)
plt.plot(history1.history['loss'], label='Loss Entrenamiento')
plt.plot(history1.history['val_loss'], label='Loss Validación')
plt.title('Evolución del Loss')
plt.xlabel('Épocas')
plt.ylabel('Loss')
plt.legend()

plt.show()
```



[20]:
```
from tensorflow.keras.models import load_model
# -------------------------
# Cargar el mejor modelo
# -------------------------
mejor_modelo = load_model('mejor_modelo_model1.keras')

# -------------------------
# Evaluar en conjunto de prueba
# -------------------------
score = mejor_modelo.evaluate(X_test, y_test)
print("Pérdida:", score[0])
print("Exactitud:", score[1])
```

```
56/56              0s 3ms/step -
accuracy: 0.7271 - loss: 0.5580
```

```
Pérdida: 0.5426663160324097
Exactitud: 0.7339761853218079
```

8. Proporcionar las predicciones del dataset de test.

[21]:
```python
from sklearn.metrics import confusion_matrix, classification_report
# Evaluate the model on the test set
loss, accuracy = model1.evaluate(X_test, y_test, verbose=0)
print(f"Test Loss: {loss:.4f}, Test Accuracy: {accuracy:.4f}")

# Make predictions and threshold at 0.5
pred_probs = model1.predict(X_test)
predtest = (pred_probs > 0.5).astype(int).flatten()  # Ensure shape (n,)

# Compute confusion matrix
cm = confusion_matrix(y_test, predtest)
print("Confusion Matrix (rows=true, columns=predicted):")
print(cm)

# Detailed classification metrics (positive class = 1)
print("\nClassification Report (positive class = 1):")
print(classification_report(y_test, predtest, target_names=['0', '1']))
```

```
Test Loss: 0.5438, Test Accuracy: 0.7249
56/56              0s 3ms/step
Confusion Matrix (rows=true, columns=predicted):
[[328 327]
 [158 950]]

Classification Report (positive class = 1):
              precision    recall  f1-score   support

           0       0.67      0.50      0.57       655
           1       0.74      0.86      0.80      1108

    accuracy                           0.72      1763
   macro avg       0.71      0.68      0.69      1763
weighted avg       0.72      0.72      0.71      1763
```

[22]:
```python
from sklearn.metrics import classification_report, accuracy_score,␣
↪precision_score, recall_score, f1_score

accuracy = accuracy_score(y_test, predtest)
precision = precision_score(y_test, predtest)
recall = recall_score(y_test, predtest)       # Sensibilidad
f1 = f1_score(y_test, predtest)
```

```python
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall (Sensitivity): {recall:.4f}")
print(f"F1 Score: {f1:.4f}") # combinacion de presicion i sensibilidad.
```

```
Accuracy: 0.7249
Precision: 0.7439
Recall (Sensitivity): 0.8574
F1 Score: 0.7966
```

[ ]:

```python

```